RELATÓRIO DE REVISÃO DE ARQUITETURA

Tennis Data Analytics

Aderência a padrões de engenharia de dados · Fase: refinamento · Setembro de 2026

3	9	4	16
críticos	moderados	menores	achados no total

Resumo executivo

O desenho segue os padrões de mercado: **arquitetura medalhão** (raw → bronze → silver → gold) no MinIO, orquestração no **Airflow**, **star schema** no Postgres e **Power BI** na visualização. As lacunas estão nas **garantias**: chaves estáveis, reprocessamento seguro, proteção de segredos e qualidade de dados. Hoje o pipeline funciona quando tudo dá certo, mas não protege o dado quando algo foge do esperado.

Pontos fortes

- Camadas bem separadas, com nomenclatura consistente (`tb_`, `dim_`, `fact_` e os prefixos `COD_`, `NUM_`, `DES_`, `SK_`).
- Star schema correto, com um fato para cada nível de detalhe (partida, torneio, temporada, ranking) e `dim_date` gerada.
- DAGs divididas por domínio e cadência (diária e semanal), com task groups legíveis.
- Retries, alertas de sucesso e falha no Telegram e logging padronizado.
- Leitura e escrita centralizadas no `PySparkHandler`.
- Deploy automatizado via GitHub Actions e diagramas de arquitetura e de modelo.

Aderência a padrões

Padrão	Status	Observação
Arquitetura medalhão	Presente	A bronze faz trabalho da silver e a raw é mutável (5, 6)
Modelagem dimensional	Parcial	Star schema correto, mas SKs instáveis e identidade por nome (1, 2)
Idempotência	Parcial	Silver e gold são idempotentes; a bronze com append não é (7)
Orquestração	Parcial	DAGs bem divididas, mas com dependência implícita e uma DAG órfã (9, 10)
Segurança	Insuficiente	Segredos no XCom e Fernet key vazia (3)

Qualidade e observabilidade	Parcial	Alertas no Telegram, sem checagem de dados (11)
Reprodutibilidade e CI	Parcial	Deploy automatizado, sem testes, carga histórica manual (13, 16)

Prioridades recomendadas

1. **Tornar as SKs determinísticas e a identidade por ID (itens 1, 2 e 9).** Hoje o dashboard pode mostrar o ranking do jogador errado. É o problema de dado mais sério, e a correção é barata.
2. **Tirar os segredos do XCom e definir a Fernet key (item 3).** Poucas linhas por DAG, com grande ganho de segurança.
3. **Fazer a carga no Postgres de forma atômica e adicionar checagens de qualidade (itens 4 e 11).** Isso protege o Power BI de tabelas vazias ou quebradas.
4. **Depois:** raw imutável, bronze sem regra de negócio, seed de correções, Datasets entre DAGs e pool do Spark (itens 5–8, 10 e 12).
5. **Por último:** padronização do código, infraestrutura e testes no CI (itens 13–16).

Achados detalhados

1 Surrogate keys instáveis — CRÍTICO

`SK_PLAYER`, `SK_TOURNEY` e `SK_ENTRY_TYPE` são geradas com `monotonically_increasing_id()` a cada overwrite, e o valor muda a cada execução. `dim_players` é refeita todo dia, mas `fact_player_ranking` só às segundas. Por isso, de terça a domingo, o ranking no Postgres pode apontar para o jogador errado.

`src/gold/dimension/dim_players.py:80 · dim_tournaments.py:94 · dim_entry.py:47`

Recomendação: Gerar a SK de forma determinística: hash da chave natural (`xxhash64(COD_PLAYER_ID)`) ou tabela de mapeamento persistente, em que só registros novos recebem uma SK nova.

2 Identidade do jogador baseada em nome — CRÍTICO

`dim_players` agrupa por nome normalizado, então homônimos viram um só jogador. `fact_player_ranking` faz join por nome. O join `ID OR ID_OLD` em `fact_player_match_stats` pode multiplicar linhas, e o `.distinct()` esconde o sintoma.

`dim_players.py:69 · fact_player_ranking.py:45 · fact_player_match_stats.py:44–58, 91`

Recomendação: Usar `COD_PLAYER_ID` como chave natural. A fonte `ATP_Database.csv` (bronze `tb_atp_players`) já traz o ID: resolver nome → ID uma única vez, na silver.

3 Segredos expostos — CRÍTICO

`AIRFLOW__CORE__FERNET_KEY` está vazia, então as Variables ficam em texto puro. A task `load_variables` devolve a senha do JDBC e as chaves do S3, e esse retorno vai para o XCom, visível na interface do Airflow.

`docker-compose.yml:16 · dags/*_dag.py (load_variables)`

Recomendação: Definir uma Fernet key. Usar Connections e lê-las dentro de cada task com hooks, sem passar credenciais pelo XCom.

4 Carga no Postgres sem atomicidade — MODERADO

`truncate=true` + `overwrite` deixa a tabela vazia ou parcial durante a carga ou após uma falha, e o Power BI pode ler esse estado.

`save_table em todos os arquivos src/gold/*`

Recomendação: Gravar em `gold_stg.<tabela>` e trocar de forma atômica numa transação, ou usar views que apontam para a versão nova.

5 Bronze com regra de negócio

MODERADO

A bronze de partidas já filtra Davis Cup, Olimpíadas etc. Ela deixa de ser uma cópia fiel da fonte, e não dá para reprocessar se a regra mudar.

`src/bronze/tb_atp_matches.py:83–93`

Recomendação: Mover os filtros para a silver. A bronze deve só tipar e acrescentar metadados (`DATE_INGESTION`, arquivo de origem).

6 Raw mutável

MODERADO

`atp_matches_{year}.csv`, `tb_ongoing_tourneys.csv` e `tb_players.csv` são sobrescritos no mesmo caminho. O histórico do que foi recebido se perde, o que impede auditoria e reprocessamento.

`bot/match_extractor.py:52–53` · `bot/player_extractor.py:9`

Recomendação: Particionar a raw por data de extração: `raw/matches/ingestion_date=YYYY–MM–DD/`.

7 Deduplicação incremental por nome e data

MODERADO

O anti-join da bronze usa `tourney_date + winner_name + loser_name`. Uma correção de grafia na fonte gera duplicata, e o `append` sem chave acumula lixo.

`bronze/tb_atp_matches.py:60–75` · `bronze/tb_atp_ranking.py:59–73`

Recomendação: Usar chave natural (`tourney_id + match_num`, `date + player_id`) com MERGE/upsert. Delta Lake ou Iceberg resolvem isso e a atomicidade no lago.

8 Correções manuais de dados no código

MODERADO

Os IDs de Davidovich, Shevchenko e De Jong e o `tourney_id` de Munich estão fixos no código e repetidos em mais de um arquivo da silver.

`silver/tb_atp_matches.py` · `tb_atp_player_match.py` · `tb_atp_players.py`

Recomendação: Criar uma seed versionada (`seeds/player_id_overrides.csv`) aplicada em um único ponto da silver.

9 Ingestão de jogadores órfã

MODERADO

`ingestion_bronze_player_dag` não é disparada por nenhuma DAG principal, e a silver de jogadores continua derivando os dados das partidas, ignorando a bronze nova.

`dags/ingestion_bronze_player_dag.py` · `src/silver/tb_atp_players.py:18`

Recomendação: Ligar a DAG ao fluxo semanal e usar `bronze/tb_atp_players` como fonte principal da silver de jogadores.

10 Dependência implícita entre DAGs

MODERADO

O fato de ranking (semanal) lê `dim_players` e `dim_date`, que são produzidas pela DAG diária. Nada declara essa dependência.

`src/gold/fact/fact_player_ranking.py:19–28`

Recomendação: Usar Airflow Datasets, ou reconstruir as dimensões conformadas numa DAG própria da qual as duas dependam.

11 Sem checagem de qualidade de dados

MODERADO

Não há validação de nulos em chaves, unicidade, integridade fato → dimensão nem contagem mínima. O join com `dim_date` é inner, então linhas podem sumir em silêncio.

`Toda a camada gold`

Recomendação: Adicionar uma task `dq_checks` após a gold (`SQLColumnCheckOperator`, Great Expectations ou Soda) que falhe a DAG.

12 Spark local dentro do scheduler

MODERADO

Cada task sobe uma JVM com 4 GB de driver no LocalExecutor, e as tasks paralelas disputam a memória da VM.

`src/utils/pyspark_handler.py:22–23`

Recomendação: Criar um pool do Airflow (`spark_pool`, 1–2 slots). Depois, avaliar `SparkSubmitOperator` ou um container Spark separado.

13 Carga histórica manual

MENOR

O fluxo é baixar localmente e depois subir à mão no MinIO, o que não é reprodutível nem orquestrado.

`bot/init_historical_*.py · README`

Recomendação: Criar uma DAG de backfill (`init_run=True`) que grave direto no bucket.

14 Código inconsistente

MENOR

Há mistura de `sys.path.append`, `from utils` e `from src.utils`; `print` no lugar de logger; o mesmo `run()` e o boilerplate de JDBC repetidos em cerca de 15 arquivos; e notebooks que duplicam `src/`.

`src/**`

Recomendação: Padronizar `from src.utils` e criar os helpers `write_to_postgres` e `run_job(load, transform, save)`. Deixar os notebooks só para exploração.

15 Views SELECT * sem função

MENOR

As views da gold só espelham as tabelas.

`dags/silver_gold_*_dag.py`

Recomendação: Dar às views o papel de camada semântica para o BI, ou usá-las na troca atômica da carga.

16 Infraestrutura e CI

MENOR

Os containers rodam como root com `chmod 777`; `airflow db init` está descontinuado; o `JAVA_HOME` é fixo em arm64; o `requirements.txt` diverge do Dockerfile; o CI não roda testes nem lint.

`docker-compose.yml · Dockerfile · .github/workflows`

Recomendação: Usar o UID do Airflow e `airflow db migrate`, um único `requirements.txt` pinado, e rodar `pytest` e `ruff` antes do deploy.

Revisão feita sobre o branch main (commit e102ff5). Números de linha referem-se a esse commit.